Glyph Editor

Customization guide: type a normal key, see your own glyph

1. What you have

A small Windows text editor written in C++. When you press `a`, the editor draws **your picture for "a"** instead of the letter. Digits, vowels and consonants that you list in one text file are replaced; every other character (space, punctuation…) stays normal.

`hello` → 

Illustration using the sample glyphs g08, g05, g12, g12, g15 (h, e, l, l, o in the demo mapping).

> **Key idea.** The editor *stores real letters* and only *draws* pictures. A saved file is ordinary text, so you can always press **F2** to see plain letters, and nothing is lost if a picture is missing.

> **Honest status.** The editor's "brain" (`core.h` : typing, deleting, selection, file reading, reading `glyphs.txt`) passed an automatic self-test. The Windows window code (`main_win.cpp`) was written carefully but **I could not compile it** in my environment (no Windows compiler). If your compiler shows an error, send me the exact message and I will fix it.

2. The folders

Path	What it is
`src/config.h`	**Your settings.** Glyph folder location, start file, colours, tab width.
`glyphs/glyphs.txt`	**The list** "key = picture". Also size settings.
`glyphs/g01.png … g18.png`	Your 18 sample glyphs, cleaned up (transparent background, same size).
`src/core.h`	The brain (plain C++, no Windows code).
`src/main_win.cpp`	The Windows window: drawing, keyboard, menus, Open/Save.
`tools/glyph_tool.py`	Cleans new pictures; packs everything into one .exe.
`build_*.bat`, `make_portable_*.bat`	One-click builds.
`tests/test_core.cpp`	Self-test of the brain.

3. Build and run (Windows)

1. Install a C++ compiler: **MinGW-w64** (via MSYS2 or WinLibs) *or* Visual Studio Build Tools.
2. Double-click `build_mingw.bat` (MSVC users: run `build_msvc.bat` from the “x64 Native Tools Command Prompt”).
3. Run `GlyphEditor.exe`. Keep the `glyphs` folder next to it. Menu **Help → About** shows which folder it used and how many pictures loaded (the demo should say 18).

4. Tell it where your glyph folder is

Open `src/config.h`. The first setting is the place reserved for you:

```
// >>> 1. PUT THE FOLDER THAT HOLDS YOUR GLYPH PNGs + glyphs.txt HERE <<<
#define GLYPH_FOLDER L"C:\\Users\\Donatien\\Documents\\MyGlyphs"
```

Every backslash is written twice. Leave `L""` to use the program’s own folder. The search order is: your folder → a `glyphs` folder next to the .exe → the .exe’s own folder. A second reserved spot, `DEFAULT_TEXT_FILE`, opens a text file automatically when the editor starts. **Rebuild after changing `config.h`.** Changes to `glyphs.txt` need no rebuild, only a restart.

5. The list: `glyphs/glyphs.txt`

One line per character: `key = picture.png`. The 18 sample glyphs are given to the letters a to r **only as a demo**. Please re-assign them to the right keys.


a = g01.png


b = g02.png


c = g03.png


d = g04.png


e = g05.png


f = g06.png


g = g07.png


h = g08.png


i = g09.png


j = g10.png


k = g11.png


l = g12.png


m = g13.png


n = g14.png


o = g15.png


p = g16.png


q = g17.png


r = g18.png

```
# lines starting with # are comments
a = g01.png
b = g02.png
s = my_s.png          # add your own: just copy the PNG into the folder
7 = my_seven.png      # digits work the same way
U+0023 = hash.png     # use U+ codes for special keys such as # or =
```

- Any character **not** listed stays normal text. That is how spaces, punctuation and unfinished letters behave.
- Want vowels only? List only `a e i o u`. Want everything? List a–z and 0–9.
- `@case_insensitive=1` makes **A** use the same picture as **a**. Set `0` and add lines such as `A = big_a.png` to give capitals their own glyphs. (On Windows, file names are not case sensitive, so name them `a.png` and `a_cap.png`.)

6. Prepare new pictures

Your pictures can be any size: the editor fits them into the character box and keeps their proportions. For the best look, clean them first (needs Python and `pip install pillow`):

```
python tools/glyph_tool.py normalize my_raw_pictures glyphs --size 96
python tools/glyph_tool.py normalize my_raw_pictures glyphs --size 96 --color 1A1AFF   #
blue ink
```

It trims the empty border, centres each glyph in a square, turns the white background transparent, and saves small PNGs. Every glyph keeps its own file name, so name the raw pictures the way you want them (`s.png`, `seven.png`...).

7. Customize the look

What	Where	Change
Box size of each character	`glyphs.txt`	`@cell_width=44` and `@cell_height=64` (bigger = bigger glyphs)
Space around each picture	`glyphs.txt`	`@glyph_padding=6`
Font for unmapped characters	`glyphs.txt`	`@font=Consolas` and `@font_px=34`
Page, text, selection, cursor colours	`config.h`	`COLOR_BACKGROUND 255, 255, 255` and the other `COLOR_*` lines (R, G, B)
Glyph colour	tool	`glyph_tool.py normalize … --color RRGGBB`
Spaces per Tab key	`config.h`	`TAB_WIDTH 4`
Window title / start size	`config.h`	`APP_TITLE`, `START_WIDTH`, `START_HEIGHT`

Because every character sits in a fixed box, text lines up like a typewriter. Narrow and wide glyphs all get the same cell.

8. Make it portable (one .exe)

Run `make_portable_mingw.bat` (or the MSVC version). It does two things:

1. `glyph_tool.py pack` reads `glyphs.txt`, collects every PNG it mentions and writes them into `src/embedded_glyphs.h`.
2. The program is compiled with `-DEMBED_GLYPHS`, so it reads the pictures from *inside itself*.

Result: **GlyphEditor_Portable.exe**, one file you can carry on a USB stick. PNG files are already compressed, so no extra compressing step is needed; 18 glyphs add only about 26 KB. After you change a glyph or `glyphs.txt`, run the portable build again.

9. Keys and menus

Key	Action	Key	Action
F2	glyphs ⇄ plain letters	Ctrl+N / O / S	New / Open / Save
Ctrl+Shift+S	Save As	Ctrl+A / C / X / V	Select all / copy / cut / paste
Arrows, Home, End	move (Shift = select)	Ctrl+Home / End	start / end of text
PageUp / PageDown	move one page	Mouse wheel	scroll (Shift = sideways)

10. If something goes wrong

Problem	Fix
A message lists pictures that could not be loaded	The file name in `glyphs.txt` does not match a PNG in the folder. Check spelling and extension; Help → About shows the folder in use.
Letters look normal instead of glyphs	That key is not in `glyphs.txt`, or the file was not found, or F2 is on plain-letters mode (the title shows “[plain letters]”).
Compiler: `wWinMain` / `WinMain` not found	MinGW needs `-municode` (already in the .bat files).
Compiler: `SHCreateMemStream` or `min` / `max` errors	Use a recent MinGW-w64 (WinLibs or MSYS2). Send me the exact error and I will adjust.
Portable build shows “Could not find glyphs.txt”	Run `make_portable_*.bat` again so the header is regenerated from your folder.

11. Limits of this first version, and next steps

- No undo/redo, no word wrap, no emoji beyond basic characters.
- Glyphs show only inside this editor. Other programs (Notepad, WhatsApp) see the real letters.
- **Android:** `core.h` is plain C++ and can be reused. An Android version would need a new window layer (a Java or Kotlin view that draws the same PNGs). Ask me for it as the next step.
- Ideas: a second “scrambled” save format, a glyph picker, bold/italic, line numbers.

Your glyph pictures stay in your own files. Nothing in this program connects to the internet.